Broadway Property Management

Developer Handoff

Rwanda-focused property management SaaS

NestJS 11 · PostgreSQL · Next.js 16 · React 19 · Tailwind 4

Generated: 2026-05-19

Table of Contents

Demo accounts (staging / local seed)

Role	Email	Password
Platform operator	platform@broadway.demo	Demo2026!
Landlord owner	owner@demo-landlord.rw	Demo2026!
Tenant	tenant@demo-landlord.rw	Demo2026!

Change passwords on any public staging URL. Seed: node backend/scripts/seed-demo-staging.js

Document version: 1.0
Last updated: May 2026
Repository: broadway-property-management (monorepo: backend/ + frontend/)

Executive summary

Broadway Property Management is a Rwanda-focused property management SaaS for landlords, tenants, accountants, lawyers, and platform operators who resell the product to multiple landlord workspaces.

Area	Stack
API	**NestJS 11**, TypeORM, PostgreSQL
Web app	**Next.js 16**, **React 19**, **Tailwind CSS 4**
Auth	JWT access + refresh tokens, role-based guards
Currency / locale	**RWF**, UPI fields, Rwanda tax trackers (not live RRA filing)
Payments (MVP)	Manual MoMo/bank **proof upload**; gateway intent is a placeholder

Primary personas

- OWNER — landlord workspace admin; full portfolio, team, settings
- ACCOUNTANT / LAWYER — scoped modules (expenses, contracts, payments review)
- TENANT — portal: pay rent (proof), view leases
- PLATFORM_OWNER — multi-client operator: activate/suspend landlords, rolled-up KPIs

MVP status: Feature-complete for pilot landlords and partner demo
filing APIs are not integrated; manual workflows and PDF exports a

---

Architecture

```
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   Browser (localhost:3001 or Vercel)                                  %
%   Next.js 16 — marketing + /dashboard/*                               %
%   /api/* % % rewrite% % %º API_PROXY_TARGET (default http://localhost:3000) %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %,% % % % % % % % % % % % % % % % % % % % % % % % % % %
                             %  JWT Bearer
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %¼% % % % % % % % % % % % % % % % % % % % % % % % % % %
%   NestJS API (port 3000 / Render)                                       %
%   Modules: auth, accounts, billing, buildings, units, tenants,          %
%   contracts, payments, invoices, expenses, compliance, analytics,       %
%   platform, dashboard, audit, notifications                             %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %,% % % % % % % % % % % % % % % % % % % % % % % % % % %
                             %  TypeORM
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %¼% % % % % % % % % % % % % % % % % % % % % % % % % % %
%   PostgreSQL (local or Neon)                                            %
%   Upload dirs: uploads/contracts, payments, expenses, ebm, rdb            %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

Concern	Detail
Ports	Backend `3000`, frontend `3001`
API proxy	`frontend/next.config.ts` rewrites `/api/:path*` !’ `API_PROXY_TARGET` (see `frontend/src/lib/api.ts`)
Auth	`Authorization: Bearer <accessToken>`; refresh via `POST /auth/refresh`
Roles	`PLATFORM_OWNER`, `OWNER`, `ACCOUNTANT`, `LAWYER`, `TENANT` — enforced with `@Roles()` + `RolesGuard`
Multi-tenancy	`accountId` on all tenant data; platform account `kind=PLATFORM` with child `LANDLORD` workspaces
Scheduling	`@nestjs/schedule` registered; rent reminder batch via `POST /payments/rent-reminders/run`

Features implemented (MVP)

Explored from backend/src, frontend/src, seeds, and partner docs.

Authentication & accounts

- Email/password signup with subscription plan selection (billing/plan-catalog.ts)
- Login, refresh, logout, forgot/reset password (email delivery requires SMTP)
- Google OAuth (POST /auth/google) when GOOGLE_CLIENT_ID set
- Microsoft OAuth (POST /auth/microsoft) when MICROSOFT_CLIENT_ID set
- GET /auth/me, workspace billing patch (accounts/me/billing)
- Team invites: POST /accounts/users, activation toggles

Billing plans

- Public GET /billing/plans — Starter / Professional / Business / Enterprise / Platform Partner (RWF pricing in catalog)
- Plan stored on account at signup; limits described in catalog (enforcement is soft / documentary for MVP)

Dashboard hub (role-specific tiles)

- GET /dashboard/hub — counts and deep links per role (dashboard.service.ts)
- Landlord: Properties, Units, Tenants, Leases, Payments, Tax, Expenses (owner/accountant), Team (owner), Annual forecast, Settings, Operations
- Platform: Clients, Finance (rolled up), Tax, Operations, Settings
- Tenant: My rent & payments, My leases
- Frontend hub: frontend/src/app/dashboard/page.tsx + department-tile.tsx

Properties, units, tenants, leases

- Buildings — name, address, UPI, property kind, usage type (buildings/)
- Units — per building, floor, unit name (units/)
- Tenants — profiles linked to units; tenant user signup; RDB certificate upload/download (tenants/)
- Contracts — PDF upload, versions, approve, rent terms; download by version (contracts/)
- Lease PDF parsing — not implemented; rent/terms entered manually on upload

Payments, proofs, invoices

- Payment settings (MoMo/bank toggles, account numbers) — owner only
- Rwanda commercial bank list (static reference)
- Manual proof flow: tenant submits MoMo SMS text and/or screenshot; owner reviews (PATCH /payments/:id/review)
- Invoices list + PDF download; rent reminder invoice generation
- RRA purchase code + EBM receipt fields on payments; EBM PDF download
- POST /payments/gateway-intent — returns AGGREGATOR_PLACEHOLDER (no live checkout)
- Webhook stub: POST /payments/webhooks/provider (public)

Tax & compliance

- Rwanda tax profile + obligation register (VAT, land/property, PIT/CIT trackers)
- CRUD obligations; summary; export obligations PDF (GET /compliance/obligations/export.pdf)
- RRA resource links — informational, not filing API
- Banner in UI: tracker only, not tax filing

Expenses

- CRUD expenses with optional receipt upload (uploads/expenses/)
- Summary endpoint for dashboards

Annual forecast

- GET /analytics/annual-forecast — Jan–Dec projection from rent + manual lines
- Owner/accountant can add/delete manual income lines

Team & settings

- Team page (owner): role counts, user list from accounts API
- Settings: payment methods, workspace/legal preferences (frontend pages wired to payments/accounts APIs)

Tenant portal

- Routes: /dashboard/portal, /dashboard/portal/pay, /dashboard/leases (tenant nav in navigation.ts)
- Pay rent: quote + proof submission

Platform (multi-client)

- GET /platform/overview — all landlord clients under platform account
- Create workspace, patch activation (ACTIVE / HOLD / SUSPENDED)
- Clients UI: /dashboard/clients

Uploads

Type	Path	API

Lease PDF

`uploads/contracts/`

`POST /contracts`, version upload

Payment proof

`uploads/payments/`

`POST /payments/proofs`

RDB certificate

(tenant)

`POST /tenants/:id/rdb-certificate`

EBM receipt

`uploads/ebm/`

payment EBM issue + download

Expense receipt

`uploads/expenses/`

`POST /expenses` multipart

Marketing & legal pages

- / home, /pricing, /about, /contact, /login
- /legal/privacy, /legal/terms
- Pricing reads plan catalog via API

PWA

- frontend/public/manifest.json + metadata.manifest in root layout
- Standalone display, theme colors; installable on supported browsers (no custom service worker in repo)

Audit

- GET /audit — filtered event log (login, uploads, edits, downloads)
- Written from controllers via AuditService

Analytics

- Overview KPIs (occupancy, collections, outstanding)
- Revenue trend, building performance, team role counts

Onboarding

- /dashboard/onboarding — setup guide (property !' units !' tenant !' lease !' reminders)

Backend MVP reference

Modules & controllers

Module	Controller prefix	Notes
`auth`	`/auth`	signup, login, OAuth, refresh, me
`accounts`	`/accounts`	me/billing, users CRUD
`billing`	`/billing`	`GET /plans`
`buildings`	`/buildings`	CRUD
`units`	`/units`	CRUD, by building
`tenants`	`/tenants`	profiles, RDB cert
`contracts`	`/contracts`	versions, approve, download
`payments`	`/payments`	settings, proofs, invoices, review, EBM
`expenses`	`/expenses`	CRUD + summary
`compliance`	`/compliance`	profile, obligations, PDF export
`analytics`	`/analytics`	overview, forecast, trends
`platform`	`/platform`	overview, clients
`dashboard`	`/dashboard`	`GET /hub`
`audit`	`/audit`	event log
`notifications`	(service only)	SMTP email, WhatsApp webhook
`invoices`	(entity + payments routes)	invoice PDF generation in payments layer

Entry: backend/src/main.ts — global ValidationPi

Environment variables

See backend/.env.example. Additional vars used

Variable	Purpose

`DB_*`, `DB_SYNCHRONIZE`, `DB_SSL`

PostgreSQL

`JWT_SECRET`, `JWT_REFRESH_SECRET`

Tokens

`PORT`

API listen (default 3000)

`NODE_ENV`

production checks

`CORS_ORIGINS`

Comma-separated allowed origins

`APP_URL`

Links in emails / redirects

`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`,
`SMTP_FROM`

Email

`WHATSAPP_WEBHOOK_URL`

Optional reminders

`GOOGLE_CLIENT_ID`

Google sign-in

`MICROSOFT_CLIENT_ID`

Microsoft sign-in

`DEFAULT_ACCOUNT_ID`

Platform scripts default

Seed & utility scripts (`backend/scripts/`)

Script	Purpose
`seed-demo-staging.js`	Platform + demo landlord + tenant, building, unit, contract placeholder, tax row
`seed-default-account.js`	Baseline account
`ensure-platform.js`	Promote user to platform owner
`promote-owner.js`	Role promotion helper
`smoke-api.js`	Quick API smoke test
`check-demo.js`, `check-tenant-user.js`, `fix-demo-contract.js`, `fix-tenant-account.js`	Demo data repairs

NOT live (integration stubs)

Integration	Status
Bank payment gateways (BPR, BK, Equity, etc.)	`PaymentGatewayService` returns `AGGREGATOR_PLACEHOLDER`, no `checkoutUrl`
MTN / Airtel MoMo APIs	Same placeholder; manual proof only
RRA e-filing API	Compliance is **tracker + PDF export** only
Live EBM issuance to RRA	Local EBM receipt file workflow only
WhatsApp	Requires `WHATSAPP_WEBHOOK_URL`
Lease PDF auto-parse	Upload storage only; manual rent fields

Frontend MVP reference

Key routes (`frontend/src/app/`)

Route	Purpose
`/`	Marketing home
`/pricing`, `/about`, `/contact`	Marketing
`/login`	Auth
`/dashboard`	Hub tiles + landlord KPIs
`/dashboard/onboarding`	Setup guide
`/dashboard/properties`, `/units`, `/tenants`, `/leases`	Core PM
`/dashboard/payments`, `/tax`, `/expenses`, `/forecast`	Finance
`/dashboard/team`, `/settings`, `/operations`	Admin
`/dashboard/clients`	Platform operator
`/dashboard/portal`, `/portal/pay`	Tenant
`/legal/privacy`, `/legal/terms`	Legal

Redirects (legacy paths): customers!’tenants, sales!’leases, finance

Components (`frontend/src/components/`)

- auth-form.tsx, google-sign-in-button.tsx

- dashboard/dashboard-shell.tsx, dashboard-chrome.tsx, dashboard-page.tsx
- department-tile.tsx, metric-card.tsx, status-banner.tsx
- ui/trust-banner.tsx, empty-state.tsx, loading-page.tsx

Environment

Variable	Purpose
`NEXT_PUBLIC_API_BASE_URL`	Direct API URL (staging/production)
`API_PROXY_TARGET`	Rewrite target for `/api` (local: `http://localhost:3000`)
`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	Google button (must match backend)

Default browser behavior: API_BASE_URL = /api (proxy-friendly fo

Session

- Tokens in localStorage via use-session.ts / auth helpers in frontend/src/lib/auth.ts

Done checklist (repo-evidenced)

& Monorepo with NestJS backend and Next.js frontend
& PostgreSQL + TypeORM entities (buildings, units, tenants, contracts, payments, invoices, expenses, tax)
& JWT auth with refresh and role guards
& Signup with plan selection; billing plan API
& Role-based dashboard hub tiles
& Properties (UPI), units, tenants, leases with PDF versions
& Manual payment proof + invoice PDFs + owner review
& Tax obligation tracker + PDF export
& Expenses module with uploads
& Annual forecast analytics
& Team user management (owner)
& Tenant portal (pay rent, leases)
& Platform multi-client overview + activation
& Audit log API
& Marketing pages + legal pages
& PWA manifest
& CORS + /api proxy for single-origin demos
& Demo seed script for staging
& DEPLOY_STAGING.md, SETUP.md, PARTNER_WALKTHROUGH.md
& render.yaml blueprint for API on Render
& Automated tests (Jest scaffold only; minimal specs)
& DB migrations (dev uses DB_SYNCHRONIZE=true)
& Live payment / RRA integrations
& i18n (Kinyarwanda/French)

Left to do (prioritized)

P0 — Production readiness

1. Set DB_SYNCHRONIZE=false; add TypeORM migrations
2. Strong secrets rotation; lock CORS to Vercel URL only
3. File storage on object store (S3/R2) instead of local uploads/ on Render
4. Health check endpoint beyond GET / hello
5. Error monitoring (Sentry) and structured logging

P1 — Integrations

1. Bank / MoMo aggregator behind PaymentGatewayService
2. RRA e-filing when API access available (keep tracker as fallback)
3. SMTP production + WhatsApp provider webhook
4. Google/Microsoft OAuth production client configuration

P2 — Product

1. Lease PDF parsing (extract rent, dates, parties)
2. Plan limit enforcement (units, seats)
3. Bulk import (spreadsheet onboarding per Business plan)
4. Kinyarwanda / French UI
5. Accountant read-only exports / report packs

P3 — Engineering

1. E2E tests (Playwright) for signup !’ proof !’ approve
2. API contract tests for critical paths
3. CI pipeline (lint, build, test on PR)
4. Rate limiting and upload virus scanning
5. Data residency / DP law documentation pack

Local development quick start

Prerequisites

- Node.js 18+
- PostgreSQL 13+
- Git

Database

```
CREATE DATABASE broadway_pm;
```

Backend

```
cd backend
cp .env.example .env
# Edit DB_* and JWT_* secrets
npm install
npm run start:dev
```

API: http://localhost:3000

Frontend

```
cd frontend
echo NEXT_PUBLIC_API_BASE_URL=http://localhost:3000 > .env.local
# Optional for proxy-only mode: omit NEXT_PUBLIC and rely on /api rewrite
npm install
npm run dev
```

App: http://localhost:3001 (proxies /api !' backend when API_PROXY_TARGET unset defaults to localhost:3000)

Demo credentials (after seed)

Run:

```
cd backend
node scripts/seed-demo-staging.js
```

Role	Email	Password
Platform operator	`platform@broadway.demo`	`Demo2026!`
Landlord owner	`owner@demo-landlord.rw`	`Demo2026!`
Tenant	`tenant@demo-landlord.rw`	`Demo2026!`

Change these passwords on any public staging U

Partner tunnel (single port)

```
npx --yes localtunnel --port 3001
```

See PARTNER_DEMO.md for Cloudflare Tunnel alternative.

Staging deployment

Full steps: DEPLOY_STAGING.md (repo root).

Layer	Service
Database	[Neon](https://neon.tech) — `broadway_pm`
API	[Render](https://render.com) — root `backend/`, `npm run start:prod`
Frontend	[Vercel](https://vercel.com) — root `frontend/`

Render env highlights: DB_SYNCHRONIZE=false, JWT_SECRET, CORS_ORIGINS, APP_URL

Vercel env: NEXT_PUBLIC_API_BASE_URL=https://<render-api>. API_PROXY_TARGET same URL

After deploy, seed from your machine with Neon credentials:

```
cd backend
node scripts/seed-demo-staging.js
```

Partner-facing walkthrough (print/PDF): docs/PARTNER_WALKTHROUGH.md

Related documentation

File	Description
`SETUP.md`	Local setup guide
`DEPLOY_STAGING.md`	Vercel + Render + Neon
`PARTNER_DEMO.md`	Tunnel-based partner demos
`docs/PARTNER_WALKTHROUGH.md`	Step-by-step staging demo script
`backend/src/billing/plan-catalog.ts`	Subscription tiers (RWF)
`frontend/src/lib/navigation.ts`	Sidebar routes per role
`render.yaml`	Render blueprint

Support contacts

For Broadway business questions, use the contact page in the dep
questions, refer to this document and the git history on the main int

End of developer handoff.